Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

20. *Fold*

`foldr`

```haskell
sumatorio :: Num a => [a] -> a  -- predefinida como sum
sumatorio [] = 0
sumatorio (x:xs) = x + sumatorio xs

todoVerdad :: [Bool] -> Bool  -- predefinida como and
todoVerdad [] = True
todoVerdad (x:xs) = x && todoVerdad xs

aplanar :: [[a]] -> [a]  -- predefinida como concat
aplanar [] = []
aplanar (xs:xss) = xs ++ aplanar xss
```

- El esquema es el mismo.

```haskell
foldr :: (elto -> res -> res) -> res -> [elto] -> res
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- La `r` significa que empieza a operar por la derecha.
- Informalmente, `foldr f v [x1, x2, x3]` ⇝ `x1 `f` (x2 `f` (x3 `f` v))`.
- Con `foldr`, todas las funciones anteriores se escriben sin recursión explícita:

```haskell
sumatorio'  xs = foldr (+)  0    xs
todoVerdad' xs = foldr (&&) True xs
aplanar'   xss = foldr (++) []   xss
```

- Con frecuencia, los tipos `elto` y `res` son el mismo, pero no siempre.

```haskell
longitud :: [elto] -> Int
longitud xs = foldr (\_ parcial -> 1 + parcial) 0 xs
```

- En este caso: `res` = `Int`; `elto` = cualquiera.

- Normalmente, el valor base `v` es el elemento neutro por la derecha del operador.
  - `foldr (+) 7 xs` es discutible.
- Otros ejemplos:

```
factorial n = foldr (*) 1 [1..n]

sumaCuadrados xs = foldr (\x parcial -> x*x + parcial) 0 xs

cuantos p xs = foldr f_parcial 0 xs
               where f_parcial x parcial =
                       if p x then 1 + parcial else parcial

pertenece y xs = foldr (\x parcial -> x == y || parcial) False xs
```

`foldl`

- La `l` significa que empieza a operar por la izquierda.
- Informalmente, `foldl f v [x1, x2, x3]` ↝ `((v `f` x1) `f` x2) `f` x3`.

```
foldl :: (res -> elto -> res) -> res -> [elto] -> res
foldl f v [] = v
foldl f v (x:xs) = foldl f (f v x) xs
```

- Ejemplo:

```
base10 :: [Int] -> Int
base10 xs = foldl (\acum x -> acum*10 + x) 0 xs
```

  - `base10 [1, 2, 3]` ↝ `123`

- `foldl` es la implementación del patrón *función auxiliar con acumulador*.

```
daLaVuelta :: [a] -> [a]
daLaVuelta = foldl (flip (:)) []
-- daLaVuelta = foldl (\acum x -> x : acum) []
```

- Esta es la versión eficiente de invertir una lista con acumulador.

```
insertar :: Ord a => a -> [a] -> [a]
insertar x [] = [x]
insertar x (y:ys) = if x <= y then x : y : ys
                              else y : insertar x ys

insSort' xs = foldl (flip insertar) [] xs
-- insSort' xs = foldl (\acum x -> insertar x acum) [] xs
```

- Cuando el operador no es asociativo, `foldr` y `foldl` pueden dar resultados distintos.
  - `foldl (-) 0 [1, 2, 3]` ↝ `((0 - 1) - 2) - 3`
  - `foldr (-) 0 [1, 2, 3]` ↝ `1 - (2 - (3 - 0))`

Con listas infinitas

- (Aprovechando la ocasión: `repeat x = x : repeat x`.)
- `foldr` puede funcionar mejor con listas infinitas:

```
  foldr (&&) False (repeat False)
= False && foldr (&&) False (repeat False)
= False
```

```
  foldl (&&) False (repeat False)
= foldl (&&) (False && False) (repeat False)
= foldl (&&) ((False && False) && False) (repeat False)
= ...
```

- Pero no siempre:

```
  foldr (||) False (repeat False)
= False || foldr (||) False (repeat False)
= foldr (||) False (repeat False)
= ...
```

```
  foldl (||) False (repeat False)
= foldl (||) (False || False) (repeat False)
= foldl (||) ((False || False) || False) (repeat False)
= ...
```

`foldl'`

```
  foldl (+) 0          [1, 2, 3, 4]
= foldl (+) (0+1)         [2, 3, 4]
= foldl (+) ((0+1)+2)        [3, 4]
= foldl (+) (((0+1)+2)+3)       [4]
= foldl (+) ((((0+1)+2)+3)+4)    []
= (((0+1)+2)+3)+4
= ...
= 10
```

```
                     foldr (+) 0 [1, 2, 3, 4]
= 1 +                foldr (+) 0    [2, 3, 4]
= 1 + (2 +           foldr (+) 0       [3, 4])
= 1 + (2 + (3 +      foldr (+) 0          [4]))
= 1 + (2 + (3 + (4 + foldr (+) 0           [])))
= 1 + (2 + (3 + (4 + 0)))
= ...
= 10
```

```
  foldl' (+) 0          [1, 2, 3, 4]
= foldl' (+) (0+1)        [2, 3, 4]
= foldl' (+) 1            [2, 3, 4]
= foldl' (+) (1+2)           [3, 4]
= foldl' (+) 3               [3, 4]
= foldl' (+) (3+3)              [4]
= foldl' (+) 6                  [4]
= foldl' (+) (6+4)               []
= foldl' (+) 10                  []
= 10
```

```
:set +s
foldl (+) 0 [1..10000000]
foldl' (+) 0 [1..10000000]
```

¿Qué *fold* uso?

situación	*fold*
El resultado es una lista hecha con `:` o `++`.	`foldr`
Lista quizá infinita, operador perezoso (`&&`, `:`).	`foldr`
Función auxiliar con acumulador (`base10`, `daLaVuelta`).	`foldl'` o `foldl`
Acumular un número a partir de una lista grande (`sum`, `cuantos`).	`foldl'`

- Por qué `foldr` para construir listas:
  - Conserva el orden:
    - `foldr (:) [] [1, 2, 3]` ↝ `1 : (2 : (3 : [])) = [1, 2, 3]`
    - `foldl (flip (:)) [] [1, 2, 3]` ↝ `[3, 2, 1]`
  - Con `++` es eficiente:
    - `foldr (++) [] [xs1, xs2, xs3]` ↝ `xs1 ++ (xs2 ++ (xs3 ++ []))`
      - Cada lista se recorre una vez.
    - `foldl (++) [] [xs1, xs2, xs3]` ↝ `(([] ++ xs1) ++ xs2) ++ xs3`
      - Lo ya construido se recorre una y otra vez.
  - Funciona mejor con listas infinitas.
- Por qué `foldl'` para acumular un número (u otros valores simples):
  - `foldr` y `foldl` dejan las operaciones pendientes hasta el final.
  - `foldl'` opera en cada paso.
  - Con listas pequeñas da igual (los ejemplos del principio usan `foldr`).

Fuera de menú

- Sirven para cualquier tipo que implemente `Foldable`.

```haskell
data ArbolBin a = ABVacio | ABHoja a
                | ABNodo (ArbolBin a) a (ArbolBin a)

instance Foldable ArbolBin where
   foldr f v ABVacio = v
   foldr f v (ABHoja x) = f x v
   foldr f v (ABNodo izq x der) = foldr f (f x (foldr f v der)) izq
```

- `foldl (+) 0 (ABNodo (ABNodo (ABHoja 2) 5 (ABHoja 7)) 3 (ABHoja 4))` ⇝ `21`
- `sum (ABNodo (ABNodo (ABHoja 2) 5 (ABHoja 7)) 3 (ABHoja 4))` ⇝ `21`
- `length (ABNodo (ABNodo (ABHoja 2) 5 (ABHoja 7)) 3 (ABHoja 4))` ⇝ `5`

Ejercicios

1. ¿Cuál es el valor de esta expresión?:
   - `take 4 (foldr (\i parcial -> i : map (+3) parcial) [] (repeat 1))`

2. Considera esta función:
   - `pertenece1 y xs = foldr (\x parcial -> x == y || parcial) False xs`

   ¿Actúa de manera perezosa, es decir, para en el momento en que encuentra el elemento buscado? ¿Y si consideramos esta otra?:
   - `pertenece2 y xs = foldr (\x parcial -> parcial || x == y) False xs`

3. La función `foldl` devuelve solo el resultado final, pero a veces interesan también los valores intermedios del acumulador. Define una función `escanear` que, como `foldl`, recorra la lista acumulando con `f` desde `v`, pero que devuelva la lista de *todos* los valores que va tomando el acumulador (incluido `v` al principio). Por ejemplo: `escanear (+) 0 [1, 2, 3]` ⇝ `[0, 1, 3, 6]`. (Esta función está predefinida como `scanl`.)

4. Define una función `fib :: Int -> Int` usando algún *fold* y sin recursión explícita.

5. En el módulo `Data.Foldable` hay una función `toList :: Foldable t => t a -> [a]`. Impleméntala a partir de alguna de las versiones de *fold*.